

Sidekick — Riepilogo Progetto

| Conversazione di brainstorming e pianificazione tecnica tra Pietro e Claude

1. L'Idea

Sidekick è un'app mobile regalo personalizzata. Un donatore la configura con foto e dediche per una persona cara. Il destinatario riceve l'app già popolata e scopre i contenuti nel tempo.

Meccaniche principali

- **Galleria** di 15 card, inizialmente tutte bloccate
- **Sblocco a timer:** una card del pool base si sblocca ogni 24 ore in modo randomico, anche a app chiusa (verificato al riavvio o ritorno in foreground)
- **Card speciali AR (3):** sbloccabili solo tramite realtà aumentata
- **Dettaglio card:** tap su card sbloccata → popup fullscreen con foto, titolo, dedica
- Le card sbloccate restano sempre consultabili nella galleria

Tipi di sblocco AR

Tipo	Meccanismo
GPS	Raggiungi coordinate specifiche entro 50m
Image Marker	Inquadra un oggetto fisico con la fotocamera
Hybrid	GPS + Image Marker combinati

Visualizzazione AR

Le card speciali appaiono come **oggetti 2.5D flottanti** nel mondo reale, con effetto di rotazione sull'asse verticale.

2. Analisi e Criticità Iniziali

Punti di forza identificati

- Nucleo emotivo solido: trasforma ricordi in esperienza interattiva
- Componente XR legata a luoghi fisici è la feature più differenziante
- Paragonabile a un **calendario dell'avvento digitale personalizzato**

Criticità risolte nel corso della conversazione

Criticità	Risoluzione
Distribuzione app one-off	APK sideloading su Android (iOS escluso dal prototipo)
Timer gacha apparentemente frustrante	Chiarito: sblocca la carta, non limita la visualizzazione
Componente AR tecnicamente complessa	Adottato stack dedicato (ARCore + Sceneform)
GDPR e privacy	Decade con architettura offline, nessun dato transita su server

3. Posizionamento e Pricing

Comparabili di mercato

- Fotolibri premium personalizzati: €30-80
- Esperienze AR custom per eventi: €150-400

Fasce di prezzo stimate

- €50-120 → posizionamento "fotolibro digitale evoluto"
- €150-300 → enfasi su AR e carattere artigianale

Consiglio strategico

Regalare le prime copie senza prezzo e documentare la reazione del destinatario: quel materiale diventa il marketing più efficace.

4. Tool e Stack Tecnico

Strumenti di progetto

Tool	Ruolo
Unity + AR Foundation	(opzione scartata)
Kotlin + ARCore	Engine principale + AR
SceneView Android (Sceneform fork)	Rendering AR 2.5D

Tool	Ruolo
Blender	Modellazione card 3D <i>(solo se Unity)</i>
Figma	UI mockup e layout
Midjourney / Adobe Firefly	Asset grafici e texture
VS Code / JetBrains Rider	IDE per Unity C# <i>(solo se Unity)</i>
Google Antigravity	IDE agentico per sviluppo Kotlin

Perché Kotlin invece di Unity

- Claude Code e Antigravity gestiscono l'intera codebase in autonomia
 - Nessun Unity Editor da assemblare manualmente
 - Build Android più diretta e progetto più leggero
 - Card 3D → semplificata a **2.5D** (visivamente quasi identica, stack molto più semplice)
-

5. Architettura dell'App (Kotlin)

Struttura dati

```
kotlin

enum class CardType { TIMER, GPS, IMAGE_MARKER, HYBRID }

data class CardData(
    val cardId: String,
    val title: String,
    val dedication: String,
    val photo: String,           // path asset
    val unlockType: CardType,
    val latitude: Double?,
    val longitude: Double?,
    val radiusMeters: Float,
    val markerImage: String?
)

data class SaveData(
    val unlockedCardIds: List<String>,
    val lastUnlockTime: String
)
```

Moduli in ordine di sviluppo

1. `CardData`, `CardType`, `SaveData` — modello dati
2. `SaveSystem` — lettura/scrittura JSON in `filesDir`
3. `CardRepository` — accesso agli asset embedded
4. `UnlockManager` — logica timer + controllo AR
5. `GalleryActivity` + adapter — griglia UI
6. `DetailFragment` — popup card sbloccata
7. `ARActivity` — GPS, `ImageMarker`, Hybrid

Contenuti

Tutti i contenuti (foto, testi, coordinate) sono **embedded nell'APK come asset statici**. Nessun backend, nessuna rete.

6. Codice Prodotto (Unity — Prima dell'Adozione Kotlin)

Il seguente codice rimane un riferimento architetturale valido, la logica è portabile in Kotlin.

`CardData.cs` — `ScriptableObject`

- Campi: `cardId`, `title`, `dedication`, `photo`, `unlockType`
- Tipi: `Timer`, `GPS`, `ImageMarker`
- Coordinate GPS e marker solo per card speciali
- **Nota:** `unlockAfterHours` rimosso su suggerimento di Pietro — la gestione del timer è responsabilità del `GameManager`, non della singola card

`GameManager.cs`

- Singleton con `DontDestroyOnLoad`
- `TryUnlockTimerCard()` chiamato all'avvio e al ritorno in foreground
- Selezione randomica tra card Timer ancora bloccate
- `TryUnlockGPSCards()` con formula Haversine per distanza
- `UnlockImageMarkerCard()` per sblocco da AR
- Evento `OnCardUnlocked` per aggiornare la UI
- Property pubblica `AllCards`

`SaveSystem.cs`

- File JSON in `Application.persistentDataPath`

- Gestione eccezioni su lettura/scrittura
- `SaveData` con lista ID sbloccati + timestamp ISO 8601

`GalleryController.cs` + `CardSlotUI.cs`

- Rebuild griglia dinamica a runtime
- Slot bloccato: sagoma nera + lucchetto overlay
- Slot sbloccato: foto + titolo, tappabile
- Popup dettaglio con pulsante AR condizionale
- Barra progresso `x / 15`

7. UI Design

Brief inviato all'AI grafica (Gemini)

Richieste: splash screen, griglia galleria, card 3D (fronte/retro), popup dettaglio, overlay AR, barra progresso, notifica sblocco.

4 versioni grafiche

Versione	Tono	Palette	Stile
Ragazzo 13-17	Energico, giocoso	Blu elettrico, verde neon, nero	Urban, flat, accenti glitch
Ragazza 13-17	Vivace, romantico	Lilla, rosa cipria, oro	Soft, illustrativo, floreale
Uomo 18+	Sobrio, premium	Blu notte, grigio antracite, rame	Minimalista, serif, texture carta
Donna 18+	Elegante, caldo	Terracotta, beige, oro rosato	Botanical, acquarello, linee morbide

Valutazione output Gemini

-  4 identità visive chiaramente distinte e coerenti
-  Versione Uomo la più riuscita
-  Ragazza 13-17 troppo adulta nello stile
-  Overlay AR poco differenziato tra versioni — richiede un secondo passaggio dedicato
-  Nome "Your Photobook" da sostituire con "Sidekick"

8. Prompt per Antigravity

Impostazioni

- **Mode:** Plan (genera piano prima del codice)
- **Model:** Claude Sonnet
- **View:** Editor (non Manager — progetto sequenziale)

Struttura del prompt

Il prompt fornito ad Antigravity include:

- Descrizione completa del prodotto
 - Specifiche delle 3 modalità AR
 - Stack tecnico esplicito (Kotlin, minSdk 26, ARCore, SceneView, Gson, no Compose)
 - Ordine di implementazione dei moduli
 - Istruzione esplicita di mostrare il piano prima del codice
-

9. Prompt Marketing (per nuova chat)

Prompt 1 — Product Marketer

Analisi di posizionamento, canali organici, storytelling, pricing psychology, comunicazione della componente AR come esperienza emotiva anziché feature tecnica.

Prompt 2 — Prospettiva Acquirente

Analisi brutalmente onesta da chi compra: trigger emotivi, obiezioni reali, giustificazione del prezzo, formato demo più efficace, profilo di chi non comprerà mai.

10. Roadmap Sintetica

- ☒ Idea e concept
- ☒ Analisi criticità e risoluzione
- ☒ Scelta stack tecnico (Kotlin + ARCore + SceneView)
- ☒ Architettura moduli
- ☒ Brief grafico → output Gemini
- ☒ Prompt Antigravity
- ☒ Prompt marketing
- ☐ Sviluppo prototipo (15 card, 3 AR)
- ☐ Bilanciamento ritmo sblocco

- [] Test su dispositivo Android
- [] Prima distribuzione (regalo reale)
- [] Valutazione feedback → pricing

Documento generato da Claude Sonnet 4.6 — Aprile 2026